

```

import java.util.ArrayList;
import java.util.Random;

public class Main {
    public static void main(String[] args) {
        // use the main to invoke the simulation
        Simulation s = new Simulation();
        s.run();
    }
}

class Simulation {
    private Random rand = new Random();

    public void run() {
        // create factories for car builds
        CarFactory usFactory = new USFactory();
        CarFactory japanFactory = new JapanFactory();
        // get singleton references to Clock and Invoker
        Clock clock = Clock.getClock();
        Invoker invoker = Invoker.getInstance();
        // initialize Staff and have them subscribe to clock published events
        ArrayList<Staff> staff = new ArrayList<Staff>();
        staff.add(new Staff("Ann", 8, 12, 4));
        staff.add(new Staff("Bob", 9, 1, 5));
        staff.add(new Staff("Cal", 10, 2, 6));
        staff.add(new Staff("Deb", 11, 3, 7));
        for (Staff s : staff) {
            clock.registerObserver(s);
        }
        // Simulate 10 days of commands
        for (int day = 1; day <= 10; day++) {
            System.out.println("Day " + day);
            clock.resetDay();
            // increment the clock and let it notify the staff of new time
            while (clock.increment()) {
                int currentTime = clock.getCurrentTime();
                System.out.println("Time now: " + currentTime + clock.getAmPm());
                // (this uses Observer to drive State updates)
                // no one works at 7 PM, otherwise handle this hour's customer
                if (currentTime == 7) {
                    continue;
                }
                // pick a random command
                Command command = getRandomCommand();
                // create a random car instance
                Car selectedCar = getRandomCar(usFactory, japanFactory);
                // pick a staff member and check their state
                int startIndex = rand.nextInt(staff.size());
                Staff selectedStaff = null;
                for (int i = 0; i < staff.size(); i++) {
                    Staff s = staff.get((startIndex + i) % staff.size());
                    if (s.canProcessCommand()) {

```

```

        selectedStaff = s;
        break;
    } else {
        System.out.println("Sorry, " + s.getName() + " is " + s.getStateName());
    }
}

// have invoker set and make the request (command)
invoker.setCommand(command, selectedStaff, selectedCar);
invoker.submitCommand();
// continue clock increments until 7 PM
}
}

// Display results by staff member
System.out.println("Results:");
for (Staff f : staff) {
    System.out.println(f.getName() + ":");
    System.out.println("  Order: " + f.getCountOrder());
    System.out.println("  Services: " + f.getCountService());
    System.out.println("  Sales: " + f.getCountBuy());
    System.out.println("  Bonus: $" + f.getBonus());
    System.out.println();
}
}

private Command getRandomCommand() {
    int commandChoice = rand.nextInt(3);
    if (commandChoice == 0) {
        return new OrderCar();
    } else if (commandChoice == 1) {
        return new ServiceCar();
    } else {
        return new BuyCar();
    }
}

private Car getRandomCar(CarFactory usFactory, CarFactory japanFactory) {
    CarFactory factory;
    if (rand.nextInt(2) == 0) {
        factory = usFactory;
    } else {
        factory = japanFactory;
    }
    int CarChoice = rand.nextInt(3);
    if (CarChoice == 0) {
        return factory.createSedan();
    } else if (CarChoice == 1) {
        return factory.createCoupe();
    } else {
        return factory.createConvertible();
    }
}
}
}

```

```

class Car {
    String name;
    int cost;
    protected static int carCounter = 0;

    public Car() {
        carCounter++;
    }

    public String getName() {
        return name;
    }

    public int getCost() {
        return cost;
    }

    Engine engine;
    Suspension suspension;
    Tires tires;

    public String getComponents() {
        return engine.getInfo() + ", " + suspension.getInfo() + ", " + tires.getInfo();
    }
}

class USCoupe extends Car {
    public USCoupe() {
        super();
        name = "USCoupe " + carCounter;
        cost = 18000;
        engine = new StandardEngine();
        suspension = new RacingSuspension();
        tires = new RacingTires();
    }
}

class USSedan extends Car {
    public USSedan() {
        super();
        name = "USSedan " + carCounter;
        cost = 13000;
        engine = new StandardEngine();
        suspension = new StandardSuspension();
        tires = new StandardTires();
    }
}

class USConvertible extends Car {
    public USConvertible() {
        super();
        name = "USConvertible " + carCounter;
        cost = 23000;
    }
}

```

```

        engine = new RacingEngine();
        suspension = new RacingSuspension();
        tires = new RacingTires();
    }
}

class JapanCoupe extends Car {
    public JapanCoupe() {
        super();
        name = "JapanCoupe " + carCounter;
        cost = 15000;
        engine = new SportEngine();
        suspension = new EconomySuspension();
        tires = new SportTires();
    }
}

class JapanSedan extends Car {
    public JapanSedan() {
        super();
        name = "JapanSedan " + carCounter;
        cost = 10000;
        engine = new EconomyEngine();
        suspension = new EconomySuspension();
        tires = new EconomyTires();
    }
}

class JapanConvertible extends Car {
    public JapanConvertible() {
        super();
        name = "JapanConvertible " + carCounter;
        cost = 20000;
        engine = new SportEngine();
        suspension = new SportSuspension();
        tires = new SportTires();
    }
}

// The Abstract Factory pattern is used here
abstract class CarFactory {
    public Car createSedan() {
        return null;
    }

    public Car createCoupe() {
        return null;
    }

    public Car createConvertible() {
        return null;
    }
}

```

```
// US Factory
class USFactory extends CarFactory {
    @Override
    public Car createSedan() {
        return new USSedan();
    }

    public Car createCoupe() {
        return new USCoupe();
    }

    public Car createConvertible() {
        return new USConvertible();
    }
}

// Japan Factory
class JapanFactory extends CarFactory {
    @Override
    public Car createSedan() {
        return new JapanSedan();
    }

    public Car createCoupe() {
        return new JapanCoupe();
    }

    public Car createConvertible() {
        return new JapanConvertible();
    }
}

// Abstract class for components
abstract class Engine {
    String name;

    public String getInfo() {
        return "";
    }
}

// Concrete class for Standard Engine
class StandardEngine extends Engine {
    public StandardEngine() {
        name = "Standard Engine";
    }

    public String getInfo() {
        return "Standard Engine";
    }
}
```

```
// Concrete class for Racing Engine
class RacingEngine extends Engine {
    public RacingEngine() {
        name = "Racing Engine";
    }

    public String getInfo() {
        return "Racing Engine";
    }
}

// Concrete class for Economy Engine
class EconomyEngine extends Engine {
    public EconomyEngine() {
        name = "Economy Engine";
    }

    public String getInfo() {
        return "Economy Engine";
    }
}

// Concrete class for Sport Engine
class SportEngine extends Engine {
    public SportEngine() {
        name = "Sport Engine";
    }

    public String getInfo() {
        return "Sport Engine";
    }
}

// Abstract class for suspension
abstract class Suspension {
    String name;

    public String getInfo() {
        return "";
    }
}

// Concrete class for Standard Suspension
class StandardSuspension extends Suspension {
    public StandardSuspension() {
        name = "Standard Suspension";
    }

    public String getInfo() {
        return "Standard Suspension";
    }
}
```

```
// Concrete class for racing suspension
class RacingSuspension extends Suspension {
    public RacingSuspension() {
        name = "Racing Suspension";
    }

    public String getInfo() {
        return "Racing Suspension";
    }
}

// Concrete class for Economy Suspension
class EconomySuspension extends Suspension {

    public EconomySuspension() {
        name = "Economy Suspension";
    }

    public String getInfo() {
        return "Economy Suspension";
    }
}

// Concrete class for Sport Suspension
class SportSuspension extends Suspension {
    public SportSuspension() {
        name = "Sport Suspension";
    }

    public String getInfo() {
        return "Sport Suspension";
    }
}

// Abstract class for Tires
abstract class Tires {
    String name;

    public String getInfo() {
        return "";
    }
}

// Concrete class for Standard Tires
class StandardTires extends Tires {
    public StandardTires() {
        name = "Standard tires";
    }

    public String getInfo() {
        return "Standard Tires";
    }
}
```

```

// Concrete class for Racing Tires
class RacingTires extends Tires {
    public RacingTires() {
        name = "Racing tires";
    }

    public String getInfo() {
        return "Racing Tires";
    }
}

// Concrete class for Economy Tires
class EconomyTires extends Tires {
    public EconomyTires() {
        name = "Economy tires";
    }

    public String getInfo() {
        return "Economy Tires";
    }
}

// Concrete class for Sport Tires
class SportTires extends Tires {
    public SportTires() {
        name = "Sport Tires";
    }

    public String getInfo() {
        return "Sport Tires";
    }
}

// The Command interface
interface Command {
    void execute(Staff staff, Car car);
}

// The class for orderCar Command
class OrderCar implements Command {

    public void execute(Staff staff, Car car) {
        staff.orderCar(car);
    }
}

// The class for ServiceCar command
class ServiceCar implements Command {
    public void execute(Staff staff, Car car) {
        staff.serviceCar(car);
    }
}

```



```

// The class for BuyCar
class BuyCar implements Command {

    public void execute(Staff staff, Car car) {
        staff.buyCar(car);
    }
}

// Observer interface
interface Observer {
    void update(int currentTime);
}

// This class will create instances of observers
class Staff implements Observer {
    private String name;
    private int countOrder;
    private int countService;
    private int countBuy;
    private double bonus;
    private int arrivalTime;
    private int lunchTime;
    private int leaveTime;
    private StaffState state;

    public Staff(String name, int arrivalTime, int lunchTime, int leaveTime) {
        this.name = name;
        this.arrivalTime = arrivalTime;
        this.lunchTime = lunchTime;
        this.leaveTime = leaveTime;
        countOrder = 0;
        countService = 0;
        countBuy = 0;
        bonus = 0.0;
        state = new NotInState();
    }

    public void orderCar(Car car) {
        countOrder += 1;
        double earned = car.getCost() * 0.03;
        bonus += earned;
        System.out.println(name + " is ordering " + car.getName() + " for $" + (int) earned);
    }

    public void serviceCar(Car car) {
        countService += 1;
        double earned = car.getCost() * 0.01;
        bonus += earned;
        System.out.println(name + " is servicing " + car.getName() + " for $" + (int) earned);
    }

    public void buyCar(Car car) {
        countBuy += 1;
    }
}

```

```
        double earned = car.getCost() * 0.02;
        bonus += earned;
        System.out.println(name + " is buying " + car.getName() + " for $" + (int) earned);
    }

    public String getName() {
        return name;
    }

    public int getCountOrder() {
        return countOrder;
    }

    public int getCountService() {
        return countService;
    }

    public int getCountBuy() {
        return countBuy;
    }

    public int getBonus() {
        return (int) bonus;
    }

    public int getArrivalTime() {
        return arrivalTime;
    }

    public int getLunchTime() {
        return lunchTime;
    }

    public int getLeaveTime() {
        return leaveTime;
    }

    public String getStateName() {
        return state.getName();
    }

    public void setState(StaffState state) {
        this.state = state;
    }

    public boolean canProcessCommand() {
        return state.canProcessCommand();
    }

    public void update(int currentTime) {
        String oldState = getStateName();
        state.updateState(this, currentTime);
        String newState = getStateName();
    }
}
```

```

        if (!oldState.equals(newState)) {
            System.out.println(name + " moved to " + newState + " at " + currentTime);
        }
    }
}

// A lazy Singleton is used here
class Invoker {
    private static Invoker invoker;
    private Command command;
    private Car car;
    private Staff staff;

    private Invoker() {
    };

    public static synchronized Invoker getInstance() {
        if (invoker == null) {
            invoker = new Invoker();
        }
        return invoker;
    }

    public void setCommand(Command command, Staff staff, Car car) {
        this.command = command;
        this.car = car;
        this.staff = staff;
    }

    public void submitCommand() {
        command.execute(staff, car);
    }
}

// The standard interface for Subject
interface Subject {
    void registerObserver(Staff s);

    void notifyObservers();
}

// An eager Singleton is used here
class Clock implements Subject {

    private static Clock clock = new Clock();
    private int timeIndex;
    private int currentTime;

    private Clock() {
        resetDay();
    }

    public static Clock getClock() {
        return clock;
    }
}

```

```

}

private static final int[] dayTime = { 8, 9, 10, 11, 12, 1, 2, 3, 4, 5, 6, 7 };

ArrayList<Staff> observers = new ArrayList<Staff>();

public void registerObserver(Staff s) {
    observers.add(s);
}

public void notifyObservers() {
    for (Staff staff : observers) {
        staff.update(currentTime);
    }
}

public void resetDay() {
    timeIndex = -1;
    currentTime = 0;
}

public boolean increment() {
    timeIndex++;
    if (timeIndex >= dayTime.length) {
        return false;
    }
    currentTime = dayTime[timeIndex];
    notifyObservers();
    return true;
}

public int getCurrentTime() {
    return currentTime;
}

public String getAmPm() {
    if (currentTime >= 8 && currentTime <= 11) {
        return " AM";
    } else {
        return " PM";
    }
}
}

// State pattern is used here
interface StaffState {
    void updateState(Staff staff, int currentTime);
    boolean canProcessCommand();
    String getName();
}

class NotInState implements StaffState {
    public void updateState(Staff staff, int currentTime) {
        if (currentTime == staff.getArrivalTime()) {

```

```

        staff.setState(new ArrivingState());
    }
}

public boolean canProcessCommand() {
    return false;
}

public String getName() {
    return "NotIn";
}
}

class ArrivingState implements StaffState {
    public void updateState(Staff staff, int currentTime) {
        if (currentTime > staff.getArrivalTime()) {
            staff.setState(new AvailableState());
        }
    }
    public boolean canProcessCommand() {
        return true;
    }
    public String getName() {
        return "Arriving";
    }
}

class AvailableState implements StaffState {
    public void updateState(Staff staff, int currentTime) {
        if (currentTime == staff.getLunchTime()) {
            staff.setState(new LunchState());
        } else if (currentTime == staff.getLeaveTime()) {
            staff.setState(new NotInState());
        }
    }
    public boolean canProcessCommand() {
        return true;
    }
    public String getName() {
        return "Available";
    }
}

class LunchState implements StaffState {
    public void updateState(Staff staff, int currentTime) {
        if (currentTime != staff.getLunchTime()) {
            staff.setState(new AvailableState());
        }
    }
    public boolean canProcessCommand() {
        return false;
    }
}

```

```
public String getName() {  
    return "Lunch";  
}  
}
```